

mAIndMap: Supporting Ideation with LLMs through a Tree-Based Interface

Donghyun Kim
allloook02@yonsei.ac.kr
Yonsei University
Seoul, Korea

Sungyong Park
parksungyong0101@yonsei.ac.kr
Yonsei University
Seoul, Korea

Jin Chun
jini0329949@gmail.com
Yonsei University
Seoul, Korea

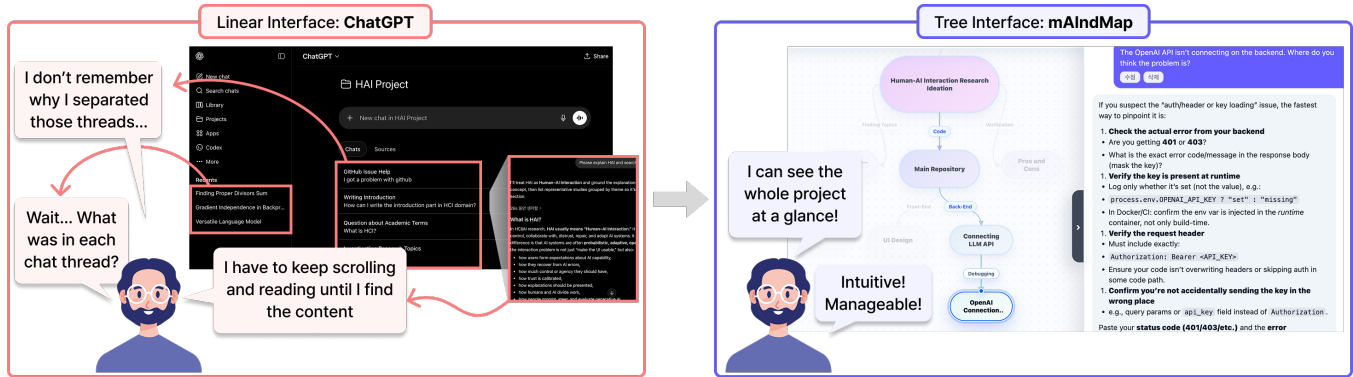


Figure 1: mAIndMap organizes LLM conversations as editable tree threads, allowing users to branch from a selected node while continuing a local chat.

Abstract

LLM-based chat systems are increasingly used for long-running sensemaking and ideation, yet most interfaces still organize conversation as a single chronological transcript. As conversations grow, this linear structure makes it difficult to revisit prior ideas, separate unrelated subtopics, and understand how different lines of thought relate. We present mAIndMap, a tree-based conversation interface that represents LLM interactions as an editable tree of conversation threads. Each node contains a local chat thread, branches support alternative directions of thought, and edge labels make relationships between threads visible. The system reconstructs model context from the selected node and its ancestor path, preserving local context boundaries while maintaining a global overview. We evaluated mAIndMap through a prototype and an exploratory within-subjects study against a linear Chat UI. Results suggest improved context focus and promising trends for idea generation and thought organization, alongside additional effort and learning cost. These findings highlight non-linear chat interfaces as active workspaces for long-running human-AI collaboration.

CCS Concepts

• **Human-centered computing** → **Graphical user interfaces**; *Collaborative interaction*; *Visualization systems and tools*.

Keywords

conversational interfaces, tree-based interface, mind-map, sense-making, human-AI collaboration

1 Introduction

Large language model (LLM)-based conversational systems are increasingly used as general-purpose partners for information seeking, writing, ideation, and problem solving. Recent large-scale analyses of ChatGPT usage suggest that practical guidance, information seeking, and writing constitute a substantial portion of real-world use cases [4]. At the same time, HCI research has shown that working effectively with LLMs is not simply a matter of asking isolated questions: users must iteratively formulate prompts, inspect outputs, revise intentions, compare alternatives, and maintain evolving task context [18]. As a result, the chat interface has become more than an input-output channel. For many knowledge tasks, it functions as the workspace in which users externalize, revisit, and reshape their thinking.

This shift makes LLM interaction closely related to sensemaking: the process of constructing and revising representations in order to answer task-specific questions [15]. In sensemaking tasks, external representations matter because they can reduce the cost of finding, comparing, and reorganizing information. Visual mapping techniques such as concept maps and mind maps similarly support users in making relationships among ideas explicit rather

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

HAI '26-1, Seoul, Korea

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2026/XX
<https://doi.org/XX>

than leaving them implicit in prose [6]. These observations are especially relevant to LLM conversations, where useful knowledge is often distributed across many turns, tentative branches, abandoned options, and partial decisions.

However, most widely used LLM interfaces still organize interaction as a linear transcript. This structure is effective for short exchanges, but it becomes increasingly fragile as conversations grow longer and more multi-threaded. Ideas that emerged at different moments are interleaved in a single chronological stream; trivial episodes sit next to high-level design decisions; and users must often scroll, search, or manually restate context in order to return to a previous line of thought.

Consider a developer collaborating with an LLM while creating a software repository. Within the same chat thread, the developer may discuss the overall architecture, decide which features to implement, revise the folder structure, debug a minor build error, and later return to earlier design tradeoffs. The flat transcript will combine all different subjects into one timeline. Splitting them into separate chats can reduce local clutter, but it also obscures how the chats relate to one another and often requires users to restate context. This creates a cognitive burden: users must remember both the content of each thread and the structure connecting threads.

Recent systems have begun to explore non-linear LLM interfaces for sensemaking, including multilevel information exploration [16], comparison of many LLM outputs [7], and canvas-based branching conversation structures [1]. Yet there remains an opportunity to treat long-running conversations themselves as user-editable knowledge structures rather than as immutable chronological logs.

To address this problem, we propose **mAIndMap**, a tree-based conversation interface for structuring LLM interactions as an editable mind map. In mAIndMap, each node represents a conversation thread rather than an individual message. Users can continue a conversation from any node, create a child node to pursue a separate direction, and return to earlier nodes without flattening all activity into a single transcript. Edges are annotated with short phrase-level labels that describe the semantic relationship between threads, such as “implementation detail,” “alternative design,” or “debugging.” By allowing users to edit nodes and edges directly, mAIndMap makes the organization of the conversation a first-class part of the interaction.

This paper makes three contributions. First, we frame long-running LLM chat as a thread-level sensemaking problem in which users must manage not only conversational content but also the relationships among evolving contexts. Second, we present mAIndMap, a tree-based interface that combines familiar chat interaction with editable mind-map structure for branching, revisiting, and organizing conversation threads. Third, we articulate design implications for non-linear LLM conversation systems, with a focus on preserving local conversational context while giving users a global overview of their thinking process.

2 Related Work

2.1 General-Purpose LLM Chat Interfaces

LLM chat services such as ChatGPT, Gemini, and Claude are typically organized around chronological chat histories. These systems have introduced mechanisms for managing conversations across

longer-term work: ChatGPT Projects can collect chats, files, instructions, and branched chats within a project workspace [14]; Gemini provides pinned and recent chats for revisiting prior activity [8]; and Claude Projects provide self-contained workspaces with their own chat histories and knowledge bases [2]. These features acknowledge that users increasingly rely on LLMs for sustained work rather than one-off queries.

Nevertheless, the dominant organizational unit remains the individual chat thread or project container. Search, folders, project workspaces, and branch creation can help users recover or separate conversations, but they do not directly expose the semantic relationships among threads. For example, users still need to remember which thread contains a design decision, which branch explored an alternative, and how a debugging episode relates to the main task. mAIndMap addresses this gap by treating the relationships among conversation threads as visible and editable objects, rather than leaving them implicit in a sidebar or linear transcript.

2.2 Structured and Visualized LLM Interaction

Recent HCI systems have explored ways to move LLM interaction beyond plain text transcripts. Graphologue converts streamed LLM responses into interactive node-link diagrams, allowing users to inspect entities and relationships in LLM-generated content [10]. Sensecape supports multilevel exploration and sensemaking over information with LLMs [16]. Other systems focus on particular aspects of the interaction process: OnGoal tracks and visualizes users’ conversational goals over time in multi-turn dialogues [5], while InteractiveKG externalizes LLM-generated reasoning as editable knowledge graphs to support inspection, correction, and trust calibration [11]. Together, these systems show that visual and structured representations can improve users’ ability to interpret, navigate, and manipulate LLM interactions.

mAIndMap builds on this line of work but differs in its primary unit of structure. Prior systems often visualize response content, extracted entities, goal progress, or reasoning state. In contrast, mAIndMap structures the conversation itself: each node represents a conversational thread that can be resumed, branched, renamed, or reorganized by the user. This conversation-centered representation is designed for long-running collaboration in which the challenge is not only understanding an individual model response, but also managing the evolving topology of the user’s own thinking with the LLM.

2.3 AI Mind-Mapping and Graph Summary Tools

Commercial AI mind-mapping tools also demonstrate growing interest in using visual structures to support comprehension and ideation. Xmind AI can transform webpages, documents, PDFs, and video links into structured mind maps [17]. MindMap AI supports generating and editing mind maps from prompts, PDFs, media files, meeting transcripts, CSVs, and webpages [13]. Mapify similarly summarizes YouTube videos, documents, URLs, podcasts, and meeting recordings into mind maps and provides editing and export features [12]. These tools are useful for condensing external materials into a navigable visual form, and they align with broader

evidence that mapping techniques can make conceptual relationships more explicit [6].

However, most AI mind-mapping tools operate primarily as content-to-map summarizers. Their nodes usually correspond to generated topics, key points, or summaries rather than to the original conversation turns and branching contexts through which ideas emerged. As a result, users may gain a compact overview while losing the fine-grained conversational provenance of an idea. mAIIndMap instead uses the tree as an interaction substrate for live LLM collaboration: users create and manipulate thread nodes as part of the conversation, preserving the link between a visual structure and the dialogue context that produced it.

3 Design Goals

Based on the limitations of linear chat histories described above, we derive five design goals for mAIIndMap. These goals are oriented toward long-running ideation and problem-solving sessions, where users may need to keep several partial lines of thought alive while still preserving the local context of each conversation.

DG1: Externalize the structure of a conversation. The interface should make the user’s evolving conversation structure visible. Instead of treating a long chat as a single chronological transcript, mAIIndMap represents each branch of work as a node in a tree. This lets users see which topics are central, which topics are side explorations, and which branches are still available to resume.

DG2: Support deliberate branching without losing continuity. Users often need to pause one line of reasoning and explore another direction, such as testing an implementation detail while preserving the larger project plan. mAIIndMap should make such branching lightweight: users can create a child node from the current node, continue the conversation there, and later return to the parent or a sibling branch.

DG3: Preserve local context boundaries. A tree view is useful only if its branches remain meaningful. Therefore, chatting in one node should not automatically expose the model to every other node in the tree. mAIIndMap follows a path-based context policy: the selected node’s current conversation is primary, while only the root-to-parent ancestor lineage is provided as background. Sibling and child branches are intentionally excluded unless the user moves to them. This boundary also keeps the prompt given to the model compact: the system does not spend tokens on unrelated branches that the user has already separated from the current line of work.

DG4: Keep the user in control of the representation. Although the system can generate short node titles and edge phrases, users should be able to revise them. In long creative tasks, the right label for a branch is often not the model’s summary but the user’s own interpretation of why that branch matters. mAIIndMap therefore supports renaming nodes, editing edge phrases, repositioning nodes, and deleting subtrees.

DG5: Reduce operational friction for iterative work. The tree should not replace chat; it should organize it. mAIIndMap keeps a familiar chat sidebar attached to the selected node, supports search across nodes and messages, and keeps a separate conversation-tree list for independent workspaces. The goal is to let users move between

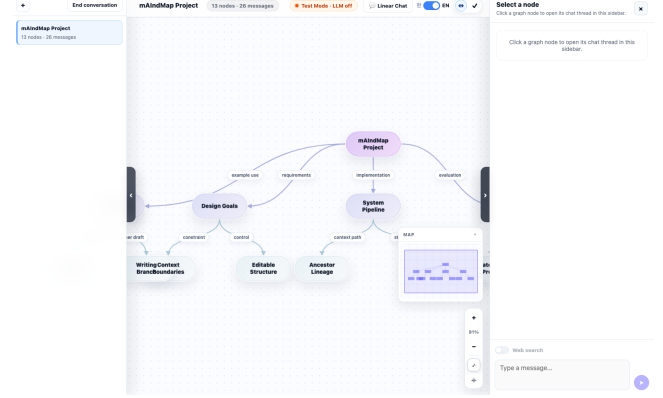


Figure 2: Example mAIIndMap workspace showing node branches, edge labels, a thread sidebar, and a chat sidebar.

overview and local conversation without repeatedly reconstructing context from memory.

4 User Scenario

Consider a developer writing an HCI paper and building a prototype at the same time. At the beginning of the session, the developer opens a root node for the project and asks the LLM to help plan the paper. The discussion quickly splits into several concerns: what design goals the interface should satisfy, how the system pipeline should be described, what kind of evaluation is reasonable, and how a user study might be framed. In a linear chat, these topics would appear in one timeline. Later, when the developer needs to find the rationale for a design decision, they would need to scroll through unrelated debugging messages and partial writing attempts.

In mAIIndMap, the developer instead turns each major direction into a child node of the project node. A *Design Goals* node captures the rationale for tree-based interaction. A *System Pipeline* node contains implementation-oriented discussion. A *Technical Evaluation* node keeps local measurements and prototype checks separate from the user-study plan. The resulting overview, shown in Figure 2, makes the state of the work visible at a glance.

The same structure also helps with detours. Suppose the developer discovers a bug while writing the system section. They can create a *Debugging Detour* child node, discuss the bug there, and return to the writing branch after the issue is resolved. The debugging conversation remains accessible, but it no longer interrupts the conceptual flow of the paper. The developer can also rename a node or edge phrase after the conversation clarifies its purpose, making the tree a user-authored representation rather than a fully automatic summary.

5 System Pipeline

mAIIndMap is implemented as a web-based tree chat prototype. The frontend provides a freeform canvas for the tree and a resizable chat sidebar for the selected node. The backend is organized around a small set of domain objects: conversation trees, tree nodes, tree edges, chat threads, and messages. Each tree node owns one chat thread, and each edge records the parent-child relation between

Algorithm 1 Context reconstruction for a selected node.**Require:** selected node v , current limit k_c , ancestor limit k_a

```

1:  $C \leftarrow \text{Recent}(v, k_c)$ 
2:  $u \leftarrow \text{parent}(v)$ 
3: while  $u$  exists do
4:    $C \leftarrow \text{Recent}(u, k_a) \parallel C$ 
5:    $u \leftarrow \text{parent}(u)$ 
6: end while
7: return  $C$ 

```

two nodes together with a short phrase-level label. Figure 3 summarizes how these objects mediate a turn of interaction: the selected tree node identifies the local chat thread, the ancestor trajectory provides bounded background context, and the resulting response is stored back in the node’s local conversation.

Workspace initialization. When a user first opens the interface, the system creates an anonymous workspace backed by a local JSON store. A workspace may contain multiple conversation trees. Each tree has a single root node that starts the main conversation for that tree. Additional nodes are created as children of an existing node, preserving a clear parent-child structure.

Node creation and labeling. When the user creates a child node, the system creates a new chat thread for that node and an edge from the parent to the child. The default node title and edge phrase can be edited immediately. After the first user prompt in a node, the system can generate a short node title and an edge phrase from the first prompt. In the prototype, this label generation is implemented as a separate title model and edge-label model so that these small metadata tasks do not have to share the same configuration as the main chat model.

Message handling and context reconstruction. When a user sends a message in the selected node, the backend retrieves the selected node, loads the recent messages from its own chat thread, and builds an ancestor context block. The ancestor context includes only the root-to-parent path of the selected node. For example, if the current node is reached through *Root* $\rightarrow$ *Design Goals* $\rightarrow$ *Context Boundaries*, the context builder includes the conversations attached to *Root* and *Design Goals*; it does not include sibling branches such as *System Pipeline* or *Technical Evaluation*. This operationalizes the tree as a context boundary rather than merely a visual decoration. Algorithm 1 summarizes this policy.

Frontend interaction loop. The frontend retrieves workspace state through the state API and renders node cards, edge paths, edge labels, a minimap, search results, and the chat sidebar. Selecting a node highlights its ancestor path and opens its local conversation in the sidebar. Figure 4 shows a selected node in the interface. The highlighted path indicates the context available to the model, while the right panel shows the local chat attached to that node.

Persistence. All tree and message state is stored as JSON in the local workspace. This keeps the implementation lightweight and makes it easy to inspect how the tree structure, chat messages, and labels are connected.

Table 1: Technical evaluation summary for synthetic tree workspaces. State KB reports serialized JSON size; Layout ms reports median layout lookup time over 200 runs.

Nodes	Edges	Msgs.	State KB	Layout ms
10	9	20	11.9	0.002
25	24	50	29.7	0.008
50	49	100	59.5	0.013
100	99	200	119.1	0.023

6 Technical Evaluation

We conducted a lightweight technical evaluation to check whether the prototype is stable and responsive enough for the planned user study. Rather than evaluating LLM answer quality, this evaluation focuses on implementation properties tied to the interface design: whether a tree workspace can support a moderate number of conversation threads, whether common frontend API actions remain responsive, and whether the context reconstruction policy reliably excludes unrelated branches.

Evaluation setup. All measurements were run locally. We generated a demo workspace with 13 nodes, 12 edges, and 26 messages through the same HTTP API used by the frontend. The generated workspace is the one shown in Figures 2 and 4.

Demo API latency. During demo construction, we recorded 55 frontend-style API calls, including node creation, title editing, edge phrase editing, message sending, state loading, and message loading. The median request latency was 10.9 ms, the mean was 12.2 ms, and the 95th percentile was 26.6 ms. These numbers should be read as application-side interaction overhead; model-generation latency is outside the scope of this evaluation and will vary by model provider, network conditions, and prompt length.

Tree scalability probe. We also generated synthetic balanced trees with 10, 25, 50, and 100 nodes. Each node contained two messages, and the tree layout service was called 200 times per size. The current layout service exposes stored node positions with shared node dimensions, so its runtime is expected to scale linearly with the number of nodes. Table 1 summarizes the results.

Context isolation probe. For the 100-node synthetic tree, the deepest selected node had depth 6. The context policy included 6 ancestor nodes in the root-to-parent path and excluded 93 non-lineage nodes. This confirms the intended behavior: context grows with the depth of the selected branch, not with the total size of the tree. The policy therefore gives users a predictable way to separate a debugging branch, a writing branch, and an evaluation branch without sending every branch to the model at once.

In this probe, a request from the deepest node would reconstruct context from the selected node and its 6 ancestors rather than serializing all 100 nodes in the workspace. This provides a prompt-footprint check: unrelated branches remain available in the workspace but are not included in the model context for that turn, which can also reduce token usage when conversations grow across multiple branches.

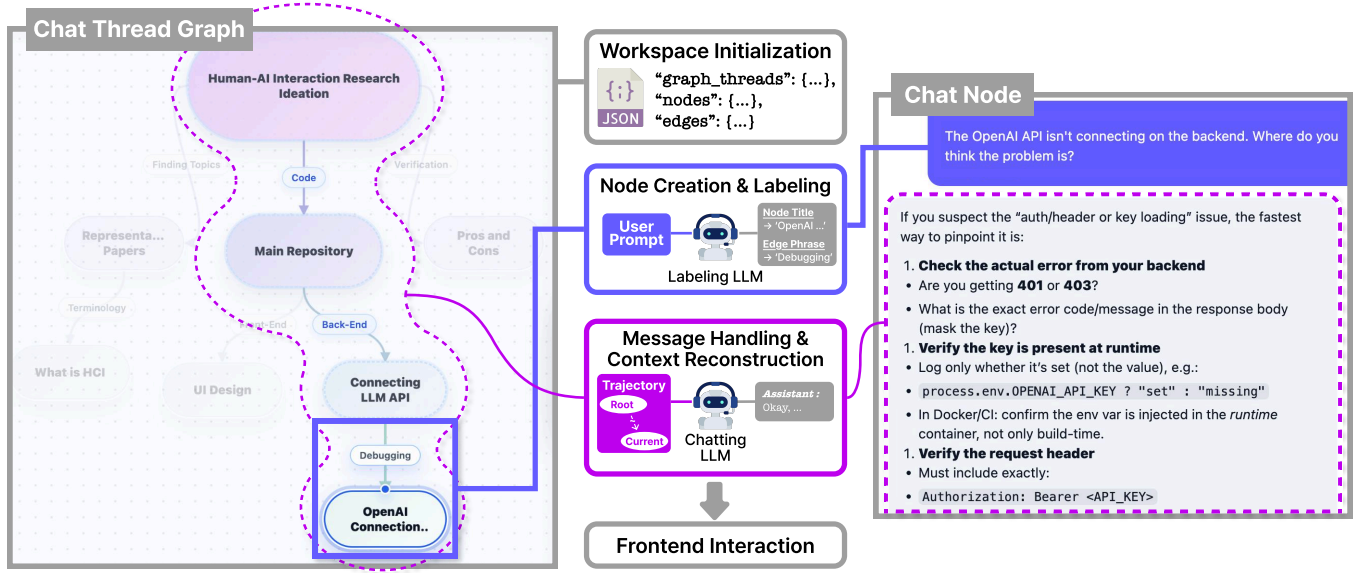


Figure 3: System pipeline of mAIIndMap. Each tree node stores a local chat thread, and the selected node determines the current prompt plus ancestor-path context used to generate the next response.

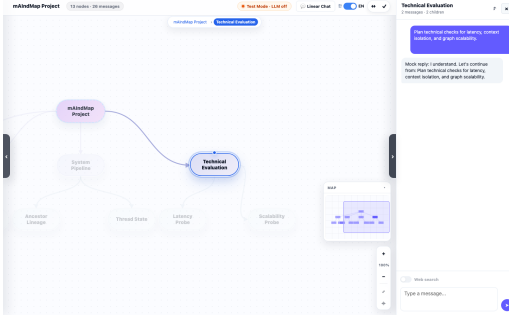


Figure 4: Selecting a node highlights its ancestor path and opens the node-local chat thread used for subsequent prompts.

7 User Study

7.1 Study Overview

We conducted a within-subjects user study to examine whether mAIIndMap supports long-running ideation more effectively than a conventional linear chat interface. Each participant used both interfaces: the proposed Tree UI and a linear Chat UI baseline. The Chat UI baseline preserved the familiar single-thread interaction style of existing LLM chat systems, while the Tree UI provided node-based conversation threads, branch creation, editable node and edge labels, and path-based local context reconstruction.

The study was designed to evaluate three questions. First, we examined whether the Tree UI helped users maintain and organize task context compared with the linear Chat UI. Second, we investigated whether the Tree UI supported ideation-related activities, such as generating diverse ideas, focusing on relevant context,

Table 2: Participant information for the nine matched pairs included in the analysis. Vis. denotes prior mind-map or diagramming-tool experience; 1st UI denotes interface order. Major abbreviations: MSE = Materials Science and Engineering, ChemE = Chemical Engineering, EEE = Electrical and Electronic Engineering, IE = Industrial Engineering, CS = Computer Science.

ID	Age	G	Major	LLM	Vis.	1st UI	Task
P1	24	M	MSE	Daily	Low	Chat	Trip
P2	23	M	MSE	Daily	Low	Chat	Study
P3	23	M	MSE	Daily	Low	Chat	Study
P4	23	F	ChemE	Daily	Low	Chat	Trip
P5	24	M	EEE	Daily	Med.	Tree	Study
P6	24	M	EEE	Daily	High	Tree	Trip
P7	23	F	MSE	Daily	Med.	Chat	Study
P8	26	M	IE	Daily	Low	Tree	Study
P9	25	F	CS	Daily	High	Tree	Trip

and organizing the flow of thought. Third, we examined usability tradeoffs introduced by the tree-based interaction model, especially whether the additional structuring operations increased learning cost or effort.

7.2 Participants

We recruited 9 participants for the within-subjects study. The matched-pair questionnaire analysis uses nine valid matched pairs. Table 2 summarizes the nine participants included in this analysis.

7.3 Tasks and Conditions

Each participant completed one of two task families. In the travel planning task, participants selected two countries or cities and planned a trip for each destination, using one interface for one

destination and the other interface for the second destination. For example, a participant might plan a trip to Seattle with the Chat UI and a trip to Shanghai with the Tree UI. In the course-study task, participants selected two courses they were currently taking and organized the course content as they would when studying, again using one interface per course. Thus, each participant worked on a single task family, while the specific topic varied across the two interface conditions. We used different topics within the same task family to reduce direct content carryover: if a participant planned the same trip or organized the same course twice, the result of the first condition could make the second condition substantially easier.

The two conditions were intended to isolate the effect of tree-based conversation structuring. Both interfaces used the same LLM backend, passed user messages to the model in the same way, and provided the same web-search capability. All model-mediated operations used GPT-5.4-nano, including chat responses and, in the Tree UI, the automatic generation of initial node titles and edge labels. The Chat UI and Tree UI therefore differed primarily in the interaction structure provided to the participant. In the Chat UI condition, all messages were accumulated in a single chronological transcript. In the Tree UI condition, participants interacted with the LLM through mAIIndMap, where each node represented a local conversation thread. Participants could create child nodes to explore subtopics, return to previous nodes, edit node and edge labels, and use the tree structure to maintain an overview of the task.

7.4 Procedure

At the beginning of the study, participants received a short introduction to the study task and the two interfaces. They then completed two task instances from the same task family, one with each interface. Each interface condition lasted 10 minutes. Interface order was randomized across participants, and each participant's first interface is reported in Table 2. After each condition, participants filled out post-task questionnaires measuring workload, usability, and perceived interface support. At the end of each condition, participants also provided an open-ended summary of the conversation they had with the AI during the task.

We collected questionnaire responses, open-ended task summaries, chat transcripts, and interaction logs. For the Tree UI, logs also preserved node-level conversation structure, allowing us to examine whether participants continued work within a selected node, created new branches, or reused local node context. For the Chat UI, logs preserved the chronological message sequence of the linear conversation.

The study used a within-subjects design because direct comparison between the Tree UI and the Chat UI was important for evaluating the contribution of tree-based conversation structuring. To reduce order effects, interface order was randomized and reported in the participant table. However, we still treat residual learning and order effects as a limitation when interpreting the results, since participants may become more familiar with the task or the study goal after completing the first condition.

7.5 Measures

We collected three types of post-task measures: adapted workload ratings, SUS-inspired usability ratings, and custom interface-support ratings. All rating items used 0–10 Likert-type scales.

First, we collected adapted NASA-TLX-style workload ratings [9]. The items included Mental Demand, Temporal Demand, Effort, Frustration, and perceived Performance. We treated Performance separately from the other workload items because higher values indicate better perceived task success, whereas higher values in Mental Demand, Temporal Demand, Effort, and Frustration indicate greater workload.

Second, we collected SUS-inspired usability ratings [3]. These items measured willingness to reuse the system, ease of use, quick learnability, perceived complexity, discomfort, and perceived learning requirement. Because our questionnaire used six items on a 0–10 scale rather than the standard ten-item SUS format, we report these as item-level usability ratings rather than as a standard SUS score.

Third, we collected custom interface-support ratings aligned with the design goals of mAIIndMap. These items measured whether the interface helped participants generate diverse ideas, stay focused on relevant context, organize their flow of thought, and reduce the number of interactions required to complete the task.

Table 3 lists the post-task questionnaire items. Participants answered these items after each interface condition.

7.6 Analysis

7.6.1 Questionnaire Analysis. Because the rating responses are ordinal Likert-type values and each participant used both interfaces, we analyzed the post-task questionnaire responses with paired Wilcoxon signed-rank tests. We report median ratings with interquartile ranges for each condition, together with the Wilcoxon statistic W and the resulting p -value. Because the study used a small within-subjects sample and short 10-minute interface sessions, we interpret the questionnaire results with caution.

7.6.2 Interaction Log Annotation and Transition Analysis. For qualitative analysis, we examined participants' open-ended summaries and interaction logs to identify recurring differences in how the two interfaces shaped conversation structure. We coded the logs at the turn-pair level, where one turn-pair consisted of a user message and the immediately following AI response. In total, the analysis included 226 turn-pairs: 105 from the Chat UI condition and 121 from the Tree UI condition.

The codebook was developed around the study's focus on context management, branching, refinement, and application-oriented ideation. It included codes for task initiation, constraint specification, linear continuation, node-local context, tree branching, parallel decomposition, context reuse, context repair, short context-dependent prompts, output refinement, application-oriented work, and external-reference risk. A single turn-pair could receive multiple labels when it served more than one interactional function. For example, a short follow-up question could be coded both as a context-dependent prompt and as context reuse if it relied on prior discussion.

After coding, we analyzed label transitions between adjacent turn-pairs. For the Chat UI, transitions were computed between

Table 3: Post-task questionnaire items. Closed-ended items used 0–10 scales; R1 was collected as a short free-response reflection after each condition.

Measure	ID	Item	Scale anchors
Workload	W1	How much mental and perceptual activity was required to perform the task?	0 = Low; 10 = High
Workload	W2	How much time pressure did you feel while completing the task?	0 = Low; 10 = High
Workload	W3	How successful did you feel in accomplishing the goals of the task?	0 = Poor; 10 = Good
Workload	W4	How hard did you have to work to achieve your performance?	0 = Low; 10 = High
Workload	W5	How irritated, stressed, or displeased did you feel while performing the task?	0 = Low; 10 = High
Usability	U1	I would like to use this system frequently.	0 = Strongly disagree; 10 = Strongly agree
Usability	U2	I think this system is easy to use.	0 = Strongly disagree; 10 = Strongly agree
Usability	U3	I think most people would learn to use this system very quickly.	0 = Strongly disagree; 10 = Strongly agree
Usability	U4	I think this system is unnecessarily complex.	0 = Strongly disagree; 10 = Strongly agree; negative item
Usability	U5	I think this system is very inconvenient to use.	0 = Strongly disagree; 10 = Strongly agree; negative item
Usability	U6	I needed to learn many things before I could use this system properly.	0 = Strongly disagree; 10 = Strongly agree; negative item
Interface support	S1	Using this interface, I was able to generate diverse and rich ideas.	0 = Strongly disagree; 10 = Strongly agree
Interface support	S2	I could focus on the relevant context without being distracted by content unrelated to the current discussion.	0 = Strongly disagree; 10 = Strongly agree
Interface support	S3	After completing the task, I felt that the overall flow of my thoughts was well organized.	0 = Strongly disagree; 10 = Strongly agree
Interface support	S4	This interface reduced the number of interactions needed to complete the task.	0 = Strongly disagree; 10 = Strongly agree
Reflection	R1	Briefly summarize the conversation you had with the AI during the task, including main topics, key ideas or decisions, and how the conversation developed.	Free response, within 100 characters

Table 4: Workload and perceived performance ratings by interface, reported as median (IQR) with Wilcoxon signed-rank tests. Higher values indicate greater workload for W1, W2, W4, and W5, and better perceived performance for W3.

Item	Tree	Chat	W	p
W1 Mental Demand	6.0 (4.0)	6.0 (3.0)	11.5	.672
W2 Temporal Demand	3.0 (4.0)	3.0 (5.0)	10.0	.260
W3 Performance	7.0 (2.0)	6.0 (2.0)	6.5	.201
W4 Effort	8.0 (2.0)	6.0 (3.0)	7.0	.235
W5 Frustration	2.0 (3.0)	3.0 (4.0)	14.5	.622

consecutive turn-pairs within the same linear conversation. For the Tree UI, transitions were computed between consecutive turn-pairs within the same node-level conversation. This produced 92 adjacent transition pairs for Chat UI and 64 adjacent transition pairs for Tree UI. We used these transition patterns to characterize the conversational rhythms encouraged by each interface.

7.7 Quantitative Results

We report median scores and interquartile ranges (IQR) for each condition, along with the results of Wilcoxon signed-rank tests ($n = 9$ matched pairs). All items used 0–10 Likert-type scales.

7.7.1 Workload and Perceived Performance. Table 4 summarises the NASA-TLX-style workload items. No item reached statistical significance at $\alpha = .05$, which is expected given the small sample size; nevertheless, several descriptive patterns are consistent with the intended design of mAIIndMap.

Mental Demand was identical at the median level (Tree = 6.0; Chat = 6.0; $W = 11.5$, $p = .672$), indicating that both interfaces imposed comparable levels of cognitive engagement. Temporal Demand showed a descriptive advantage for Tree UI (Tree Mdn =

Table 5: SUS-inspired usability ratings by interface, reported as median (IQR) with Wilcoxon signed-rank tests. U4–U6 are negatively worded; lower values indicate better usability for those items.

Item	Tree	Chat	W	p
U1 Frequent Use	7.0 (3.5)	6.0 (3.0)	10.0	.497
U2 Ease of Use	7.0 (5.0)	7.0 (1.0)	9.0	.196
U3 Quick Learnability	8.0 (2.0)	8.0 (1.0)	18.0	.652
U4 Unnecessary Complexity	2.0 (4.5)	3.0 (2.0)	14.5	.620
U5 Inconvenience	2.0 (2.5)	3.0 (2.0)	17.5	.944
U6 Learning Requirement	3.0 (4.0)	2.0 (3.0)	19.5	.820

3.0 vs. Chat Mdn = 3.0; $W = 10.0$, $p = .260$); while the medians are equal, the distribution of signed ranks leans toward lower temporal demand in Tree UI, consistent with the tree structure reducing the need to scroll through or reconstruct a linear transcript. Perceived Performance was higher for Tree UI at the median (Tree Mdn = 7.0 vs. Chat Mdn = 6.0; $W = 6.5$, $p = .201$), suggesting a descriptive trend toward greater perceived task success. Effort was higher in Tree UI (Tree Mdn = 8.0 vs. Chat Mdn = 6.0; $W = 7.0$, $p = .235$), reflecting the additional work of creating nodes, organising branches, and interpreting the tree structure. Frustration was lower in Tree UI (Tree Mdn = 2.0 vs. Chat Mdn = 3.0; $W = 14.5$, $p = .622$), suggesting that the tree structure may have reduced the stress associated with recovering context in long-running tasks.

7.7.2 Usability and Learnability. Table 5 summarises the SUS-inspired usability items. No item reached statistical significance at $\alpha = .05$.

Participants rated Tree UI higher on willingness to reuse the system (Tree Mdn = 7.0 vs. Chat Mdn = 6.0; $W = 10.0$, $p = .497$), suggesting that participants perceived value in the tree-based interface despite its additional interaction cost. Chat UI was rated

Table 6: Custom interface-support ratings by interface, reported as median (IQR) with Wilcoxon signed-rank tests.
^{*} $p < .05$; [†] $p < .10$.

Item	Tree	Chat	W	p
S1 Idea Generation	7.0 (2.5)	6.0 (2.0)	6.0	.091 [†]
S2 Context Focus	9.0 (1.5)	6.0 (3.0)	1.0	.027 [*]
S3 Thought Organisation	8.0 (2.0)	6.0 (2.0)	3.0	.061 [†]
S4 Interaction Efficiency	7.0 (3.0)	7.0 (2.0)	16.5	.570

comparably or higher on ease of use and quick learnability, with both conditions reaching a median of 7.0 and 8.0 respectively. This result is expected: linear chat interfaces are already familiar to most LLM users, whereas Tree UI introduces new operations such as node creation, branch navigation, and edge-label interpretation. For the negatively worded items (U4–U6), lower values indicate better usability. Perceived complexity, inconvenience, and learning requirement were comparable or slightly lower for Chat UI, suggesting that Tree UI introduced a modest learning cost that did not, however, eliminate participants’ willingness to reuse the system.

7.7.3 Interface Support for Ideation and Context. Table 6 summarises the custom interface-support items. This category showed the strongest evidence for Tree UI, including one statistically significant result.

Context Focus (S2) showed the largest and only statistically significant difference across all items (Tree Mdn = 9.0 vs. Chat Mdn = 6.0; $W = 1.0$, $p = .027$). This result directly corresponds to mAIIndMap’s path-based context design: by allowing users to work within a selected node while excluding unrelated sibling branches from the active context, Tree UI appears to have helped participants focus on the current topic without being distracted by previous or parallel lines of discussion. Idea Generation (S1) and Thought Organisation (S3) both showed descriptive trends in favour of Tree UI, approaching but not reaching conventional significance thresholds ($W = 6.0$, $p = .091$ and $W = 3.0$, $p = .061$, respectively). These patterns suggest that the tree structure may have helped participants keep multiple directions of thought available and externalise relationships among ideas, though the small sample size limits the strength of these conclusions. Interaction Efficiency (S4) showed no descriptive difference (Tree Mdn = 7.0 vs. Chat Mdn = 7.0; $W = 16.5$, $p = .570$), suggesting that the primary benefit of Tree UI is not reducing the raw number of interactions but making interactions more structured and easier to revisit.

7.8 Qualitative Results

7.8.1 Open-Ended Task Summaries. Participants’ open-ended summaries suggest that the Tree UI encouraged them to describe their work in terms of structure. Several participants referred to branches, categories, or step-by-step decomposition when summarizing their process. For example, one participant described proceeding by dividing the task into branches according to topic, while another described analyzing the task by category and then moving into more specific details. Another participant summarized a learning task as a progression from a core concept to related concepts and then to problem generation.

These responses suggest that participants were not only using the Tree UI as a visual display, but also becoming aware of the structure they were creating. The interface appears to have supported a more explicit representation of how subtopics related to one another. In contrast, Chat UI summaries more often resembled chronological lists of what happened in the conversation. Participants tended to describe the sequence of topics or outputs rather than the structure connecting them.

The open-ended summaries also reveal an important limitation of the Tree UI. Some participants experienced node creation and branch management as additional work. For these participants, the tree did not automatically reduce burden; instead, it required them to actively decide how to divide the conversation. This explains why Tree UI received higher Effort ratings and lower ease-of-use ratings than Chat UI in the descriptive mean summaries. The Tree UI therefore offers a conditional advantage: it can make long-running ideation more structured and revisitable, but only when users understand and adopt the tree as part of their thinking process.

7.8.2 Interaction Log Transition Patterns. The interaction-log transition analysis showed that the two interfaces produced different conversational rhythms. Table 7 summarizes the dominant patterns observed in the coded logs.

In the Chat UI condition, the strongest pattern was a linear refinement loop. Participants commonly asked a question, received an answer, refined that answer, and then continued with another follow-up in the same chronological thread. This pattern reflects both a strength and a limitation of the linear transcript. Chat UI made it easy to keep asking follow-up questions and deepen a single line of work, but conditions and subtopics were absorbed into the chronological flow rather than being separated into visible task units.

In the Tree UI condition, the dominant pattern was node-based local work. After participants created or entered a branch, they often continued within that node by reusing local context, refining outputs, and moving into application-oriented work such as planning, designing, analyzing, or generating study materials. In this sense, a branch acted as a local workspace: it provided a bounded context in which the participant could continue a subtask without returning to the entire conversation history.

The transition patterns also clarify how short or underspecified prompts worked differently across the two interfaces. In the Chat UI, short follow-ups tended to attach to the preceding chronological flow. In the Tree UI, similar prompts tended to attach to the currently selected node. This distinction helps explain why local context boundaries matter: the Tree UI can make brief instructions more interpretable by giving them a visible node-level context.

Taken together, the coded interaction patterns suggest that Chat UI extended conversations along a time axis, whereas Tree UI deepened work within spatially separated nodes. Chat UI supported linear answer refinement, while Tree UI supported persistent node-based workspaces in which branches could preserve local context and sustain downstream application work.

7.9 Overall Findings

Overall, the study provides preliminary evidence that mAIIndMap can support structured ideation and context management while

Interface	Dominant pattern	Interpretation
Chat UI	Follow-up questioning repeatedly alternated with answer refinement.	Participants tended to advance work along a single chronological thread, making it easy to continue one line of thought but harder to separate conditions and subtopics into visible units.
Chat UI	New constraints were usually folded back into the ongoing conversation.	The linear transcript supported flexible incremental specification, but constraints accumulated inside the same time-ordered stream.
Tree UI	Participants repeatedly reused and refined context inside the current node.	The node acted as a bounded workspace that helped participants keep a subtask active without returning to the entire conversation history.
Tree UI	Branches were followed by node-local work, refinement, and practical application.	Branching did not merely categorize topics; it created a new workspace where participants could continue planning, solving, designing, or analyzing.
Tree UI	Short follow-up prompts were interpreted through the currently selected node.	The visible node context narrowed the interpretation of underspecified instructions and made brief follow-ups easier to attach to the intended subtask.

Table 7: Representative qualitative transition patterns identified from coded interaction logs for the Chat UI and Tree UI conditions.

introducing additional operation costs. Context Focus showed the strongest quantitative signal ($p = .027$), with marginal trends for Idea Generation and Thought Organization; other workload and usability measures were non-significant.

The qualitative results provide converging evidence for the same interpretation. The coded transition patterns showed that Chat UI primarily supported a linear refinement loop: participants continued one chronological stream by asking follow-up questions and refining prior answers. Tree UI, by contrast, supported node-based work loops: participants reused local context, refined outputs, and carried out application-oriented work within bounded branches. This difference suggests that the Tree UI's advantage is not simply that it displays a tree, but that it changes how conversational context is organized and reused during the task.

These results align with mAIIndMap's goals of externalizing conversation structure, preserving local context boundaries, and making branches revisitable. They also suggest a tradeoff between immediate familiarity and structured long-term usefulness: linear chat is easier to start using, whereas Tree UI requires more active structuring but can turn branches into persistent local workspaces for long-running ideation.

8 Discussion

8.1 From Conversation History to Local Workspaces

The study suggests that the main value of mAIIndMap is not simply that it visualizes a conversation as a tree. Rather, the tree changes how users continue the conversation. In the Chat UI, follow-up prompts are attached to a single chronological transcript, so context reuse often means continuing the previous flow. In the Tree UI, follow-up prompts are attached to the currently selected node, so context reuse becomes tied to a visible local workspace. This distinction is especially important for long-running ideation tasks, where users may need to move between subtopics without flattening them into one transcript.

This interpretation is directly supported by the only statistically significant result across all questionnaire items: S2 Context Focus (Tree Mdn = 9.0 vs. Chat Mdn = 6.0; $W = 1.0$, $p = .027$). Participants rated their ability to stay focused on the current discussion markedly higher when using Tree UI, and this gap was large enough to be detected despite the small sample size. The open-ended summaries reinforce this finding: participants who used Tree UI described their work in terms of branches and categories, while Chat UI participants more often described a chronological sequence of outputs.

This also clarifies why branching matters. A branch is not only a visual category or a post-hoc summary of previous messages. It can become a place where users continue a subtask, refine outputs, and apply previous ideas. For HCI systems that support LLM-based thinking work, interface structure should be treated as part of the interaction context, not only as a way to display generated content after the fact.

8.2 The Cost of Explicit Structure

The results also show that explicit structure introduces interaction costs. Tree UI requires users to decide when to create a node, how to name or interpret a branch, and when to return to an earlier node. These actions can make the conversation more revisitable, but they also require additional effort compared with a familiar linear chat interface. This trade-off is reflected in the data: W4 Effort was descriptively higher for Tree UI (Mdn = 8.0 vs. 6.0), and some participants described node creation and branch management as additional work in their open-ended summaries.

This trade-off helps explain why Tree UI can improve perceived context focus and show trends toward better thought organisation (S3: Mdn = 8.0 vs. 6.0; $W = 3.0$, $p = .061$) while receiving comparable or slightly lower usability ratings. The ease-of-use medians were identical (U2: Mdn = 7.0 for both conditions), but the directional pattern in negatively worded items (U4–U6) suggests a modest learning cost. Notably, Tree UI's higher effort did not translate into lower willingness to reuse the system (U1: Mdn = 7.0 vs. 6.0), suggesting that participants found the additional work worthwhile.

This does not mean that tree-based interfaces should remove manual structuring entirely. Manual structure is part of what lets users express their own understanding of the task. However, future versions of mAIIndMap should reduce unnecessary friction around low-level operations, for example by suggesting branch points,

offering lightweight node-title edits, or making the current context boundary more legible before a user sends a prompt.

8.3 Design Implications for Non-Linear LLM Interfaces

Our findings point to three design implications, each grounded in both quantitative and qualitative evidence.

Non-linear LLM interfaces should make the active context boundary visible. Users need to know which prior messages and branches will shape the next response. The significant S2 result ($p = .027$) suggests that when this boundary is made explicit—as in mAIIndMap’s path-based context policy—users are meaningfully better able to focus on the current discussion without distraction from unrelated threads. The interaction log analysis reinforces this: short follow-up prompts in Tree UI attached to the selected node rather than the full transcript, making brief instructions more interpretable.

Branching should support continued work, not only organization. If a branch is created, the interface should make it easy to refine, apply, and revisit the work inside that branch. The S3 Thought Organisation trend ($W = 3.0$, $p = .061$) and the log-based observation that participants moved into application-oriented work within nodes both suggest that branches can serve as persistent local workspaces. This is more than a navigational convenience: it is a structural mechanism that lets users externalise the relationships among ideas rather than leaving them implicit in a linear transcript.

Systems should support different levels of user commitment to structure. The results show that some participants embraced the tree as a thinking tool, while others experienced node creation as overhead. The conditional nature of Tree UI’s advantage—visible in the higher Effort ratings alongside the significant Context Focus result—suggests that future designs should accommodate users who prefer lightweight suggestions they can accept, ignore, or edit, rather than requiring all users to actively curate a tree from the outset.

9 Limitations and Future Work

Our evaluation has several limitations. First, all model-mediated operations used GPT-5.4-nano, including chat responses, node-title generation, and edge-label generation. This limits claims about the absolute quality of the generated content. However, because both conditions used the same model configuration, time limit, and tool access, the comparison primarily reflects differences in conversation structure rather than model capability.

Second, the user study should be interpreted as a small-scale exploratory evaluation. We recruited 9 participants, but the quantitative analysis used nine valid matched pairs with complete responses. Most questionnaire items did not reach statistical significance, with the clearest exception being Context Focus. The small sample and 10-minute condition length likely limited statistical power and captured first-use impressions more than long-running ideation or cumulative sensemaking. Future work should use larger samples, longer sessions, and longitudinal tasks to examine whether the observed trends become stronger as conversations grow over time.

Third, the two interface conditions used comparable but not identical task instances. This reduced direct content carryover: planning

the same trip or organizing the same course twice could make the second condition artificially easier. At the same time, differences in topic difficulty or participant familiarity may still have affected the results. Future studies should use stronger counterbalancing and topic assignment procedures to separate interface effects from task-topic effects.

Fourth, the linear Chat UI baseline implemented the core single-thread chat interaction but did not reproduce the full feature set of commercial LLM chat systems, such as image-based interaction, built-in branching, project or folder organization, search, and richer history management. Thus, the comparison should be read as an evaluation of tree-based conversation structuring against a basic linear chat baseline, not as a complete comparison against all capabilities of modern production chat interfaces.

Finally, the interaction-log analysis remains exploratory. The labelled transition patterns help explain how the two interfaces shaped user behavior, but future work should more fully document the coding protocol, coder agreement process, and normalized transition counts by participant or session length.

10 Conclusion

This paper presented mAIIndMap, a tree-based conversation interface for organizing long-running LLM interactions as editable conversation threads. Instead of treating chat history as a single chronological transcript, mAIIndMap lets users branch from any node, revisit local threads, and maintain a visible overview of how their ideas relate. Through a prototype implementation and exploratory user study, we found that this structure helped participants stay focused on the relevant context and showed promising trends for ideation and thought organization, while also introducing additional effort and learning cost. These findings suggest that non-linear LLM interfaces should make active context boundaries visible and support branches as persistent local workspaces, not merely as post-hoc summaries of conversation history. More broadly, mAIIndMap points toward LLM chat systems that help users shape, revisit, and externalize their evolving thinking over time.

Acknowledgments

The authors used LLM-based tools, including ChatGPT and Codex, as writing advisors for paper structure and phrasing, translation and polishing, and agentic coding support for the prototype, LaTeX source, figures, and analysis artifacts. The authors reviewed and verified all LLM-assisted outputs and take full responsibility for the final content, claims, analyses, references, and code.

References

- [1] Rifat Mehreen Amin, Alperen Adatepe, Daniela Fernandes, Daniel Buschek, and Andreas Butz. 2026. Conversations in Space: Structuring Non-Linear LLM Interactions on a Canvas. arXiv:2605.15848 [cs.HC] <https://arxiv.org/abs/2605.15848>
- [2] Anthropic. 2026. What Are Projects? <https://support.anthropic.com/en/articles/9517075-what-are-projects> Accessed 2026-05-31.
- [3] John Brooke. 1995. SUS: A quick and dirty usability scale. *Usability Eval. Ind.* 189 (11 1995).
- [4] Aaron Chatterji, Thomas Cunningham, David J Deming, Zoe Hitzig, Christopher Ong, Carl Yan Shan, and Kevin Wadman. 2025. *How People Use ChatGPT*. Working Paper 34255. National Bureau of Economic Research. doi:10.3386/w34255
- [5] Adam J Coscia, Shunan Guo, Eunye Koh, and Alex Endert. 2025. OnGoal: Tracking and Visualizing Conversational Goals in Multi-Turn Dialogue with

- Large Language Models. In *Proceedings of the 38th Annual ACM Symposium on User Interface Software and Technology (UIST '25)*. ACM, 1–18. doi:10.1145/3746059.3747746
- [6] Martin Davies. 2011. Concept mapping, mind mapping and argument mapping: what are the differences and do they matter? *Higher Education* 62 (2011), 279–301. <https://api.semanticscholar.org/CorpusID:44953977>
- [7] Katy Ilonka Gero, Chelse Swoopes, Ziwei Gu, Jonathan K. Kummerfeld, and Elena L. Glassman. 2024. Supporting Sensemaking of Large Language Model Outputs at Scale. arXiv:2401.13726 [cs.HC] <https://arxiv.org/abs/2401.13726>
- [8] Google. 2026. Find and Manage Your Recent Chats in Gemini Apps. <https://support.google.com/gemini/answer/13666746> Accessed 2026-05-31.
- [9] Sandra G. Hart and Lowell E. Staveland. 1988. Development of NASA-TLX (Task Load Index): Results of Empirical and Theoretical Research. In *Human Mental Workload*, Peter A. Hancock and Najmedin Meshkati (Eds.). Advances in Psychology, Vol. 52. North-Holland, 139–183. doi:10.1016/S0166-4115(08)62386-9
- [10] Peiling Jiang, Jude Rayan, Steven P. Dow, and Haijun Xia. 2023. Graphologue: Exploring Large Language Model Responses with Interactive Diagrams. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST '23)*. ACM, 1–20. doi:10.1145/3586183.3606737
- [11] Minchan Kim, Yanjie Zhao, Jaeseong Ju, Jaeun Seo, and Hyunwoo Park. 2026. InteractiveKG: Transparent and user-controllable knowledge graph reasoning. *Information Visualization* (04 2026). doi:10.1177/14738716261435574
- [12] Mapify. 2026. AI Summarizer and Mind Map Generator for PDFs, Videos, and Webpages. <https://mapify.so/> Accessed 2026-05-31.
- [13] MindMap AI. 2026. Create Mind Maps Online for Free. <https://mindmapai.app/> Accessed 2026-05-31.
- [14] OpenAI. 2026. Projects in ChatGPT. <https://help.openai.com/en/articles/10169521-chatgpt-projects> Accessed 2026-05-31.
- [15] Daniel M. Russell, Mark J. Stefik, Peter Pirollo, and Stuart K. Card. 1993. The cost structure of sensemaking. In *Proceedings of the INTERACT '93 and CHI '93 Conference on Human Factors in Computing Systems* (Amsterdam, The Netherlands) (CHI '93). Association for Computing Machinery, New York, NY, USA, 269–276. doi:10.1145/169059.169209
- [16] Sangho Suh, Bryan Min, Srishti Palani, and Haijun Xia. 2023. Sensecape: Enabling Multilevel Exploration and Sensemaking with Large Language Models. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology* (San Francisco, CA, USA) (UIST '23). Association for Computing Machinery, New York, NY, USA, Article 1, 18 pages. doi:10.1145/3586183.3606756
- [17] Xmind. 2026. AI Mind Map: Instantly Create Mind Maps with Xmind's AI Features. <https://xmind.com/ai> Accessed 2026-05-31.
- [18] J.D. Zamfirescu-Pereira, Richmond Y. Wong, Bjoern Hartmann, and Qian Yang. 2023. Why Johnny Can't Prompt: How Non-AI Experts Try (and Fail) to Design LLM Prompts. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems* (Hamburg, Germany) (CHI '23). Association for Computing Machinery, New York, NY, USA, Article 437, 21 pages. doi:10.1145/3544548.3581388